Using Knowledge Graphs to Test for the Presence of Hallucinations in Closed RAG-based and CustomGPT-based LLM Systems

Granville "Randy" Miller
The Prospective Group
Randy.Miller@tpgcorp.com

Abstract

Many companies and government agencies are building large language model (*LLM) systems for proprietary or specialized uses with sensitive data. These closed systems are not permitted to access the internet as there are concerns about data leakage. The absence of internet access limits the data to information contained in the vector database which is used to implement retrieval augmented generation (RAG) or the ingested data in a Custom Generative Pre-trained Transformer (GPT). These systems aim to present authoritative responses and avoid hallucinations, factually inaccurate responses to prompts. In sensitive government and enterprise environments, hallucinated responses can lead to serious consequences, including misinformation, policy errors, or breaches of trust, making their detection and prevention a critical priority. We use knowledge graphs to represent the data and applied structured patterns to predict areas where prompts are likely to generate hallucinations in closed systems. Given the very large number of possible prompts for such a system, being able to target the testing of these systems is extremely important. Once issues are discovered, these graphs can also help developers fix issues where data is missing, erroneous, or must be enhanced using data curation.*

Keywords: prompt coverage, knowledge graphs, hallucinations, data curation, closed systems

1. Introduction

Many companies and government agencies are building closed generative AI systems for proprietary or specialized use. These systems utilize information collected in a vector database to augment or replace their Large Language Model (LLM) through Retrieval Augmented Generation (RAG) or a Custom Generative Pre-trained Transformer (GPT). To prevent data leakage or poisoning, these proprietary closed systems are prevented from accessing information beyond what is provided to them in their LLM and through RAG [Jiménez]. In contrast to these closed systems, open systems like Microsoft's Co-Pilot may dynamically search and access constantly changing information that it finds relevant to any prompt on the internet.

Closed systems are valuable because their vector databases contain a finite and manageable amount of information. In closed systems where we provide the data, knowledge graphs can play a significant role in representing and managing the finite information available. A Knowledge Graph (KG) is a structured representation of information where entities (such as people, places, concepts) are nodes, and relationships between them are labelled edges [Kejriwal]. In a closed system, this graph can effectively encapsulate the connections within the provided data, allowing the data to be represented with some certainty of its existence.

Since the system operates with a given limited amount of data, a knowledge graph provides a way to model and visualize relationships which might otherwise appear as a black box in the LLM or vector database. The data and its relationships can be more straightforwardly inferred through the interconnected nature of the graph. In testing our systems, we created knowledge graphs to understand how the system retrieves information, by not only drawing from exact matches but also from the deeper context of relationships within the data. In open systems, such a representation would be difficult due to the rapid data change and the amount of data that is potentially accessed. In closed systems, even with larger amounts of data, knowledge graphs or even partial knowledge graphs (knowledge graph fragments) allowed us to understand where data relationships existed and more importantly, where they did not. We found that when LLM-based systems lack the data to respond to a prompt, they are more likely to hallucinate.

Typically RAG-based and CustomGPT systems are built around a given topic or domain. Testing these systems becomes a matter of understanding the boundaries of the information (e.g. system boundaries)

URI: https://hdl.handle.net/10125/111535
978-0-9981331-9-5
(CC BY-NC-ND 4.0)

[Armour] and creating prompts (test cases) to cover the knowledge graph derived from the curated documents inside the boundaries. These prompts should test for missing entities and relationships within the system boundaries that often cause hallucinations. Limiting the amount of surface area to test greatly reduces the potential prompt space (possible test cases) and thus the number of test cases required to test the functionality of the system. Prompt coverage is then measured to ensure that proper testing is done.

2. The Approach

Our approach builds upon KnowGPT to enhance testing. The KnowGPT approach describes a knowledge graph prompting framework for translating prompts and the LLM system responses into KGs [Zhang]. The KnowGPT approach focuses on creating knowledge graphs to enhance the technology used in LLM solutions to reduce hallucinations. KnowGPT creates a single knowledge graph from prompts and responses in a set of prompt/response pairs. This ability to generate knowledge graphs from individual prompts is key to our approach.

However, while KnowGPT focuses on the technology side, our approach emphasizes data quality. Our approach focuses on testing to improve the quality of data in these closed systems. While our test methods can uncover technology issues such as the system's inability to access data that is in the vector database or CustomGPT, our focus is improving the data quality with additional documents, web pages, or metadata by utilizing data curation. Knowledge graphs visually present the existing and missing entities and relationships to both testers and developers.

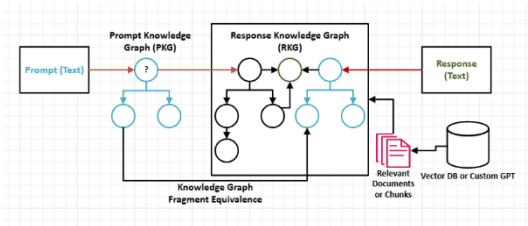

Figure 1. KG Matching for Prompts/Responses.

In contrast to KnowGPT, we separate the prompt knowledge graph (PKG) from the response knowledge graph (RKG). We utilize the RKG to create PKGs and then text-based prompts which are added to our test suites. Equivalence of the knowledge graph fragments in the RKG is used to both match prompts in the form of PKGs to likely responses (from PKG to RKG see figure 1) and generate the PKGs (from RKG to PKGs see figure 2) that allow us to predict and test for possible hallucinations. This knowledge graph approach also allows us to calculate the prompt-based test metric, prompt coverage.

In the context of testing RAG-based or CustomGPT systems, we used knowledge graphs to:

1. Identify missing entities or relationships (holes) in the data, which can then be addressed to improve the system's authoritativeness or accuracy.
2. Expand the breadth and depth of the knowledge by identifying areas in the knowledge graph where likely prompts will fall off the graph (missing references or additional curated data).
3. Create test cases (prompts) in areas of the system likely to hallucinate, providing verification and validation (and bugs) to the development team (prompt coverage).
4. Provide concrete feedback to the data curation team on why the hallucination occurred so they can continuously update the data to maintain its relevance and accuracy (data curation) [Chang].

Data incompleteness is a major cause of hallucinations (see Section 7). By using knowledge graphs, testers can systematically assess the completeness and correctness of the information that RAG-based and CustomGPT systems use, leading to more authoritative and reliable outputs. The steps above identify and measure tests for data incompleteness commonly found in creating RAG-based and CustomGPT systems.

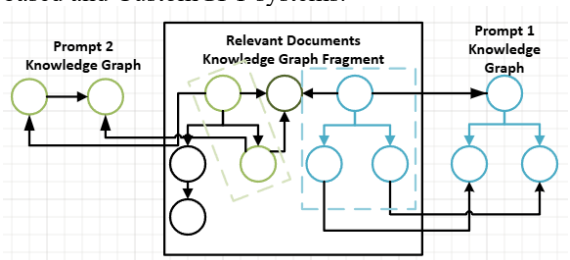

Figure 2. Creating Prompt Coverage

Prompt coverage is a metric akin to test coverage and refers to the systematic evaluation of how thoroughly a set of test prompts exercises the knowledge and reasoning capabilities of the RAG-based or CustomGPT-based solution. Analogous to test coverage in software engineering, where code paths are measured for execution during testing, prompt coverage measures how well the input prompts probe the semantic, factual, and functional breadth of the data (via responses). This includes assessing coverage across key domains, entity types, relationships, known facts, and reasoning patterns. The goal is to ensure that prompts are not overly clustered around a narrow topic set, leaving critical

knowledge areas untested where overlooked gaps can lead to hallucinations or incorrect responses.

Creating a set of prompts for comprehensive coverage of a scoped information area involves leveraging our knowledge graphs to systematically generate or select prompts across all relevant content dimensions. For example, each node and edge in a knowledge graph can be used to create one or more fact-based prompts, while combinations of related entities can be used to craft more complex reasoning tasks. This structured approach enables testers to ensure that both surface-level facts and deeper semantic relationships are exercised.

Furthermore, prompt coverage is tracked (e.g., percentage of entities/relations tested) to identify blind spots and guide iterative prompt development. As with code testing, the goal is not just quantity of prompts, but targeted, high-utility prompts that reveal gaps in data grounding, reasoning fidelity, and model robustness.

3. Test Method

The algorithm outlined below describes the steps for systematically testing closed RAG-based or CustomGPT systems for hallucinations using knowledge graphs. The goal is to create structured, explainable test cases (prompts) and analyze model responses against an authoritative reference (the knowledge graph) to detect factual errors. Each step builds on the last to transform curated data into actionable test inputs and evaluation metrics.

1. Understand the domain test boundary (see Section 8):

Define the operational and informational limits of the system. This includes the domains covered, the types of prompts expected, and the responses the system is designed to provide. The domain test boundary informs what should and should not be tested and mirrors the function of a potentially very limited system prompt in guiding acceptable behavior. A domain test boundary can be a paragraph, a document, a subject area, or a set of subject areas.

2. Determine the System Type and Source Scope (see Section 6):

Identify whether the system is RAG-based or CustomGPT-based. RAG systems rely on retrieval from semantically embedded chunks, while CustomGPTs use curated data uploaded directly or fine-tuned. Determine whether the source material is a single document or multiple documents, since this affects the granularity and construction of the knowledge graph.

3. Establish the Target Level of Authoritativeness (see Section 9):

Decide the expected level of factual grounding. This influences how detailed and complete the knowledge graph must be and what threshold will be used to flag hallucinations (e.g., complete factual alignment vs. reasonable paraphrase).

4. Create Knowledge Graph (KG) Fragments from Source Documents (see Sections 4 and 5):

Use LLM tools or subject matter experts to extract entities and relationships from each document. These KG fragments represent the building blocks of the system's knowledge. Maintain provenance metadata for each node and edge to ensure traceability.

5. Compose the Response Knowledge Graph (see Section 5):

Merge the KG fragments into a unified graph using schema alignment, entity resolution, and relation normalization. This complete or partial response KG becomes the canonical reference for identifying hallucinations and generating test prompts.

6. Perform Knowledge Graph Completion (KGC) to Identify Gaps (see Section 7):

Use structural heuristics (e.g., degree of node connectivity), schema rules, or embedding-based link prediction to find potential missing entities or relationships. These gaps represent areas where hallucinations are more likely to occur and where data curation may be needed.

7. Analyze for Hallucination Patterns (see Section 10):

Review the KG for known hallucination-prone configurations such as relation fabrication, entity inflation, overgeneralization, and contextual distortion. Use these patterns to prioritize risky test regions and guide prompt formulation.

8. Generate Prompt Knowledge Graphs and Corresponding Prompts (see Sections 2 and 8):

Transform edges and nodes from the response KG into structured prompts. Include variations to test inverse relations, synonyms, and ambiguous contexts. The system prompt should be included when generating response knowledge graphs, as it influences how responses are framed.

9. Calculate the Prompt Coverage Metric (see Section 2):

Assess how much of the knowledge graph is being exercised by the prompt set. Utilize the sets of entities and relationships in the RKG corresponding to prompts to calculate prompt coverage, the number of relationships covered divided by the total number of relationships. Iteratively add prompts to close coverage gaps.

10. Run Prompts and Check for Hallucinations (see Sections 9 and 11):

Submit each prompt to the system and transform the response into a knowledge graph. Compare the

generated response graph with the reference graph using equivalence checking (structural or semantic). Log any mismatches as hallucinations, rank their severity, and flag for curation or further investigation.

4. Creating RAG-based or CustomGPT Solutions

To understand our white box approach to test case creation, we begin by describing the method used to create RAG-based or CustomGPT knowledge bases. The data curation process for RAG-based or CustomGPT-based systems begins with data discovery and collection. This involves identifying and gathering relevant source material such as internal documents, research papers, policy manuals, web pages, PDFs, and structured data repositories. The goal is to collect high-quality, domain-specific content that reflects the knowledge needed for the system's task. During this data discovery phase, documents are inventoried and evaluated based on relevance, authority, timeliness, and comprehensiveness. For web content, crawling or scraping tools may be used, while enterprise content might come from SharePoint, file servers, or content management systems.

The next step is preprocessing and transformation, where the collected documents are cleaned and standardized. This involves removing boilerplate text (e.g., headers, footers, navigation links), normalizing encoding, fixing formatting issues, and extracting core content. HTML pages are parsed to isolate meaningful content. During this step, text may be segmented into logical chunks, such as paragraphs or sections, suitable for embedding or retrieval. This is especially important in RAG-based systems as chunk size impacts retrieval quality.

Following preprocessing, the documents undergo semantic enrichment and metadata tagging. Metadata such as document title, author, creation date, and topical categories are added to aid in indexing and filtering. Some pipelines include named entity recognition (NER), key phrase extraction, or topic modeling to automatically enrich the content with semantic tags. This additional layer of metadata enhances the system's ability to retrieve the most contextually relevant passages when answering queries or generating content.

Once documents are processed and enriched, they are indexed in a vector database or retrieval system. Each chunk of text is embedded using an encoder, such as Bidirectional Encoder Representations from Transformers (BERT) [Devlin] or OpenAI embeddings, to create a numerical representation in high-dimensional space. These embeddings allow for semantic similarity matching. In RAG systems, these embeddings are queried during inference to retrieve relevant chunks, which are then passed as context to the LLM. For CustomGPT systems, embeddings may also be used to support retrieval-enhanced fine-tuning or grounding to trusted data.

All of the documents and web pages used in these systems can be described and structured using knowledge graphs. A knowledge graph represents information as nodes (entities) and edges (relationships), providing both a human-readable and machine-interpretable framework for understanding document content. Each document creates a knowledge graph fragment of the complete system knowledge graph. Composing knowledge graph fragments into a single knowledge graph involves integrating multiple partial representations, each potentially adding new entities and relationships, into a unified, coherent structure.

5. Knowledge Graphs

Creating a knowledge graph begins with identifying the core entities and relationships present within the dataset used by the closed LLM or RAG-based system. These entities may include people, organizations, documents, locations, events, or abstract concepts, depending on the system's domain. Relationships are then defined to connect these entities in meaningful ways, such as affiliation, authorship, hierarchy, or temporal sequence. LLMs are particularly good at extracting knowledge graphs from unstructured sources using techniques such as NER and relationship extraction. The goal in this stage is to convert domain knowledge into a human-readable list of entities and relationships. Once extracted, the entities and relationships can be graphed to create a structure that faithfully captures both explicit and inferable facts.

Composing knowledge graphs begins by aligning the fragments through schema matching. Equivalent classes and properties across fragments are mapped to a common ontology or vocabulary [Shvaiko]. Next, entity resolution is performed to identify and merge duplicate or semantically equivalent nodes that may have been independently created in different fragments. Relationships are reconciled to ensure consistency and prevent redundancy or conflict (overridden terms). However, since LLMs are non-deterministic, relationship names may not be unique. The integration must also preserve provenance metadata, so the source of each fact remains traceable. This approach is especially valuable in closed systems where data curation is incremental and distributed, allowing the test team to build targeted fragments and

later integrate them into a full knowledge graph as coverage expands.

Authoritative LLM systems (see Section 9) require a nearly complete coverage of the domain and thus a nearly complete knowledge graph. A complete graph attempts to represent all known entities and relationships within the scope of the internal knowledge base (ontology). Such graphs are especially valuable in tightly scoped systems, such as those supporting a very precise subject area such as an organization's policies or a critical business process, where the information is finite and well-defined. These comprehensive graphs provide full visibility into the system's knowledge landscape and can be used for auditing the data for accuracy and up-to-datedness in addition to hallucination detection.

However, in many practical situations, particularly in large-scale or evolving systems, creating a complete knowledge graph may be infeasible due to data volume, complexity, or incompleteness. In these cases, knowledge graph fragments, subsets of the graph focusing on a specific domain, entity type, or relationship group, may be used instead. Fragments are more manageable and can be targeted toward high-risk or high-priority areas, such as those known to frequently cause hallucinations or user confusion. Selection of these knowledge graph fragments can follow the chunk selection strategy of the RAG-based system. Knowledge graph fragments are created only for the chunks selected as the system will only use this information in its formulation of the response. These targeted graphs can still offer high value in testing and refinement processes, as they allow localized assessment of the system's knowledge quality.

Both complete knowledge graphs and fragments can be created manually by subject matter experts or through the LLM. Manual methods offer the highest accuracy but may be resource-intensive, while automated methods offer scalability but may require post-processing to correct errors or inconsistencies. Regardless of the method, integrating these graphs into the development and testing lifecycle provides a powerful toolset for diagnosing knowledge gaps, structuring evaluation prompts, and informing data curation strategies. Ultimately, the decision to create full or partial graphs depends on the use case, system maturity, and available resources, but even partial graphs can significantly enhance system reliability and trustworthiness.

6. Single Papers and More

Knowledge graphs generated from a single paper aim to structure the key concepts, relationships, and entities within that specific document. These graphs typically represent a semantic map of the paper's content, identifying how ideas, processes, and terminologies relate within a constrained context. Because they are derived from a single source, their scope is inherently limited but highly precise. This approach allows for a much smaller number of targeted test cases. However, since the graph reflects only one paper's information, it may lack broader contextual relevance or connections to related works. Single papers provide useful opportunities to discover hallucinations, especially with prompts on the boundaries (entities with missing but expected relationships) of the knowledge graph.

In contrast, a retrieval-augmented generation (RAG) system divides a corpus, often consisting of multiple documents, into "chunks," which are typically small text segments stored in a vector database. At query time, relevant chunks are retrieved based on semantic similarity to the input prompt, then passed to a language model for generation. These chunks may be difficult to stitch together as a single knowledge graph since they rely on vector embeddings for relevance and retrieval. Chunks may provide broader coverage than a single-document knowledge graph and since each new prompt brings a new set of chunks. The documents behind the chunks make for a better knowledge graph mechanism to test.

CustomGPTs, meanwhile, can integrate a wide variety of information, either uploaded directly or embedded through fine-tuned instructions and tools. The information ingested by a CustomGPT is also generally unstructured or semi-structured, ranging from PDF documents to website data. This information is processed by the model contextually rather than through persistent structured representations like graphs or indexed chunks. CustomGPTs can be guided to behave consistently across sessions using system instructions and can simulate domain expertise, but they do not inherently organize information relationally (as in a knowledge graph) or by embedding-based retrieval (as in RAG). Therefore, the documents used to feed a CustomGPT are the sources for the response knowledge graph.

7. Finding Data Incompleteness Using KGs

Knowledge graph completion (KGC) is the process of inferring and predicting missing information in a knowledge graph, such as absent entities, relationships, or facts [Chen]. The incompleteness of knowledge graphs is a widespread issue due to manual or semi-automatic construction methods, which often fail to uncover all relevant facts

or latent connections [Wang]. KGC addresses this by estimating missing parts in triples, combinations of head entity (source), relation, and tail entity (target), through various techniques. The set of triples, both existing and missing, create a strong basis for creating prompts as test cases.

KGC along with the domain test boundary allows testers to perform link prediction, entity prediction, and relation prediction, all aimed at enhancing the structural and semantic integrity of the prompt knowledge graphs. In practice, finding data incompleteness using knowledge graphs begins by identifying structural gaps in the graph topology. For example, if the top-level organization node is present without any associated lower organizational units, it may indicate that the data curation for the system lacks the reporting structure for that organization. By querying for nodes with only one or no connections, testers can surface weakly linked areas of the graph, areas where a real-world user prompt is likely to produce hallucinated or incomplete answers. Visual inspection and automated coverage analysis tools can be used to highlight "thin" regions of the KG, helping prioritize data augmentation efforts.

Another effective method is the use of schema-based expectation checking. If a predefined ontology specifies that every “Policy Document” must have a “Date Issued,” “Author,” and “Subject Area,” the absence of one or more of these fields in a given document’s KG representation constitutes a semantic gap. This structured incompleteness detection ensures that entities are not only present but also contain the necessary context and attributes expected in the domain, thereby increasing the RAG-based or CustomGPT system’s reliability when asked about them.

Embedding-based completion techniques can be applied to estimate missing links and suggest data that may be missing [Wang]. If two entities often co-occur in similar contexts across different documents but lack a direct connection in the KG, the model may suggest a plausible relation (e.g., “reports_to,” “owns,” or “published_by”). Prompts that test synonyms among the KG relationships may also be added to the test suites to see how the system responds when prompts test for relationships among entities using different wording.

Knowledge graph fragments may also provide multiple relationship/target entity pairs for the same source/relationship. This is to be expected as the RAG-based or CustomGPT system provides non-deterministic responses. Multiple relations/target entity pairs may exist for a given relationship for any given source entity.

8. Impact of the System Prompt

The system prompt is a hidden or background instruction that guides how a RAG-based or CustomGPT system should behave when responding to user input. Unlike the user prompt, which is visible and changeable by the user, the system prompt is set by developers or platform designers to establish the model’s tone, rules of engagement, or objectives, such as remaining helpful, avoiding certain types of content, or following a particular logic. In some implementations, this prompt might include task-specific context, constraints, or example formatting, deeply influencing the model’s response patterns.

The system prompt also provides guardrails for the types of prompts the model will respond to. A similar concept exists in use case modeling where the system boundary defines the scope of the system’s functionality. The system boundary explicitly separates what lies inside the system or what the system is responsible for from the external actors. This boundary ensures clarity in responsibilities, inputs, outputs, and expected behavior.

Similarly, in LLM systems, the system prompt acts as a form of cognitive boundary, a set of implicit guardrails that shapes how the model interprets user inputs and constrains its behavior. Just as a system boundary in a use case diagram limits what functions are modeled or tested, the system prompt often limits what kinds of responses the system will generate, what tone it adopts, what rules it follows, and how it prioritizes information. In both cases, these boundaries serve to reduce ambiguity, focus functionality, and enforce constraints that ensure the system behaves in alignment with its intended purpose.

Our test case target, the domain test boundary, forms a third boundary. This boundary determines the area to be tested, and is usually within (but not always) that created by the system prompt. We can form and use these smaller boundaries to incrementally test our system, one area at a time.

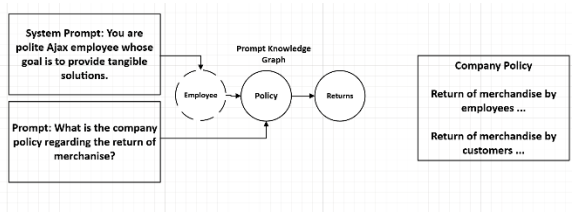

Figure 3. System Prompt Impact.

Additionally, when generating knowledge graphs from user prompts, the system prompt can subtly or explicitly shape how the AI interprets the user's intent and structures the information. For example, a system

prompt might instruct the model to "extract entities and their relationships in a scholarly tone" or "represent all causal relationships explicitly." This affects not only what the model deems important enough to extract but also how it represents relationships (e.g., whether it prefers hierarchical, associative, or temporal connections). As a result, two identical user prompts could yield different knowledge graphs depending on the underlying system prompt.

In practice, this influence means that the system prompt acts as a cognitive framework for the LLM System, shaping how knowledge is organized and represented. If the system prompt emphasizes precision and taxonomy, the resulting knowledge graph may be dense, formal, and highly structured. If it emphasizes summarization or storytelling, the graph might favor narrative-linked nodes rather than rigid ontological structures. Thus, the design of the system prompt is a critical, though often invisible, factor in shaping the fidelity, bias, and utility of the RAG-based or CustomGPT knowledge representations.

9. Authoritativeness

Authoritativeness in closed LLM systems refers to the system's ability to provide responses that are not only accurate, but also supported by trustworthy and verifiable data. In sensitive domains, such as legal, financial, and government applications, ensuring that outputs are grounded in vetted sources is paramount. Since closed RAG and CustomGPT systems operate over a fixed, curated dataset, authoritativeness depends entirely on the quality, completeness, and traceability of the ingested data.

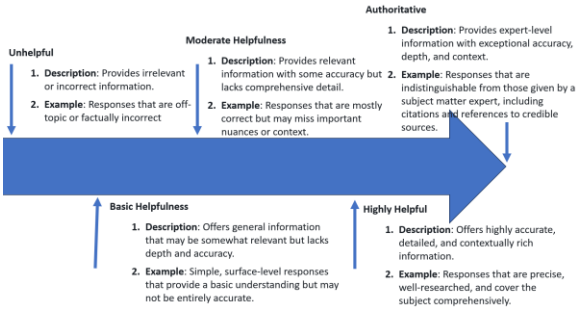

Figure 4. Levels of Authoritativeness.

Knowledge graphs contribute significantly to enhancing authoritativeness by explicitly representing the provenance and connections among facts. For instance, linking a policy regulation to its issuing agency, publication date, and authoritative source document allows users to trace any LLM-generated claim back to its origin. This traceability is crucial for auditability and trust. Furthermore, graph structures help expose areas where authoritative backing is weak or ambiguous, prompting data curators to resolve these gaps before they compromise the system's credibility.

In practice, authoritativeness can be tested by aligning LLM outputs with the knowledge graph. If a response cannot be mapped to a known fact or structure in the KG, it is likely non-authoritative and should be flagged. This alignment also enables automated explainability mechanisms where users can be shown the specific source documents that justify a particular answer, further reinforcing user trust in the system's responses. Authoritativeness influences the need for a given level of prompt coverage in testing.

10. Patterns of Hallucinations from KGs

Hallucinations, factually inaccurate responses to prompts, occur in Large Language Model (LLM) systems when a model confidently generates a response with no basis in its training data [Bouchard]. There are many reasons for the creation of hallucinations including lack of knowledge, missing mechanisms to allow verification of the factual accuracy of the responses, fictional or inaccurate content in the LLM, and inability to express uncertainty with the response. Finding hallucinations in the absence of data is difficult (but not impossible).

Knowledge graphs based on curated data provide a structured lens through which common hallucination patterns in closed RAG-based or CustomGPT systems can be identified and categorized. By mapping both system responses and the underlying curated data into graph structures, we have detected where the model's outputs deviate from the known, verified facts. These deviations often fall into recognizable patterns. Each pattern is associated with different types of knowledge gaps or misalignments between the LLM's generative capabilities and the available data.

One common pattern is relation fabrication (RF), where the model invents a connection between two known entities that do not exist in the knowledge graph. For instance, if "Agency X" and "Initiative Y" exist as separate nodes in the KG but have no direct or indirect connection, and the model states that "Agency X launched Initiative Y," this hallucination can be precisely flagged by the absence of a corresponding edge. Relation fabrication typically arises when the model generalizes from patterns seen elsewhere in the data or training corpus, leading to erroneous inferences in a closed system context.

Another pattern is entity inflation (EI), where the system introduces plausible-sounding entities that are not present in the KG at all. These hallucinations often involve made-up subunits, titles, or personnel, such as

claiming that "Department Z has a Chief Data Innovation Officer" when no such entity exists in the curated dataset. Because knowledge graphs serve as a reference for all authorized entities in the domain, hallucinations of this type are easily exposed by KG queries that fail to locate the mentioned node.

A subtler pattern is contextual distortion (CD), where the model retrieves correct entities but misrepresents the relationships or context in which they appear. For example, if the KG shows that a policy was issued in response to a specific event, but the model attributes it to a different, unrelated cause, the hallucination stems from a misalignment in contextual associations. This type of error is particularly dangerous because it often sounds plausible and may even use correctly sourced entities, but presents an inaccurate narrative that undermines trust and traceability.

Lastly, overgeneralization (OG) is a pattern where the model extrapolates too broadly from specific facts in the KG. If the KG contains a policy that applies to a single agency or user group, and the LLM response implies that it applies universally, this misapplication constitutes a hallucination. Knowledge graphs help identify overgeneralization by clarifying the scope and constraints of each node and relationship. Once testing reveals overgeneralization, augment the system with curated data, targeted metadata, or a focused system prompt for that domain to reduce drift and keep responses properly scoped. Overgeneralization is the most common form of hallucination as it represents the system attempting to be helpful but lacking the exact information necessary to create an exact response.

11. Knowledge Graph Equivalence

Knowledge graph equivalence refers to the condition in which two knowledge graphs represent the same underlying information or semantics, even if they differ in structure, formatting, or labeling. This concept is important in contexts such as system testing, knowledge integration, version comparison, and matching of PKGs in the RKG. Equivalence may be exact, where all nodes and edges correspond one-to-one, or semantic, where different representations encode the same meaning despite differences in terminology or graph topology. Determining equivalence is essential when comparing a generated response graph to a source knowledge graph, especially for detecting hallucinations or validating model outputs.

There are several dimensions of equivalence. Structural equivalence implies that the graphs are isomorphic. There exists a mapping between nodes and edges that preserves relationships and labels exactly. This is often used in controlled test cases where graphs are derived from the same ontology and data sources. Semantic equivalence, by contrast, recognizes that two graphs may differ syntactically but express the same facts. For example, "Person A works for Company B" is equivalent to "Company B employs Person A." Detecting this kind of equivalence may require reasoning over synonyms, inverse relations, or domain-specific ontologies to determine that the meaning is preserved.

Equivalence testing becomes especially relevant when comparing knowledge graphs extracted from LLM responses with those built from authoritative data. For example, if a prompt response is converted into a knowledge graph and compared against the "ground truth" KG from curated sources, equivalence checking reveals whether the LLM has maintained factual integrity. Non-equivalence between the graphs, structural or semantic, signals possible hallucination, misinterpretation, or incomplete model understanding.

Finally, knowledge graph equivalence plays a vital role in merging and aligning knowledge graphs from multiple sources. In large-scale systems, KGs are often built in fragments and updated independently. To integrate them without redundancy or contradiction, equivalence must be assessed at both the entity and subgraph levels. Tools and algorithms for entity resolution, schema matching, and graph canonicalization help reconcile different graphs into a unified, coherent structure. Whether used for evaluation, integration, or reasoning, graph equivalence is foundational to ensuring that knowledge representations remain consistent, meaningful, and verifiable across different systems and stages of processing.

12. An Example

To illustrate the algorithm in practice, we apply it to a bounded case study constructed from the three paragraphs in this paper that describe the KnowGPT prompt framework. Since the paper only summarizes KnowGPT and refers to external work for full details, the available information is intentionally limited. For the purposes of this example, we treat KnowGPT as the subject domain under test, acknowledging that a complete knowledge graph would require additional sources.

Step 1 (see Section 3) establishes the scope of analysis: we define KnowGPT as the domain test boundary. Step 2 identifies the data source, three paragraphs from a single document (this paper), to be incorporated into a RAG-based system. Step 3 concerns authoritativeness; in this example, we set it

aside, recognizing that this paper alone cannot provide authoritative coverage of the KnowGPT framework. Steps 4 and 5 then produce the response knowledge graph (RKG) for the three paragraphs (see Figure 5).

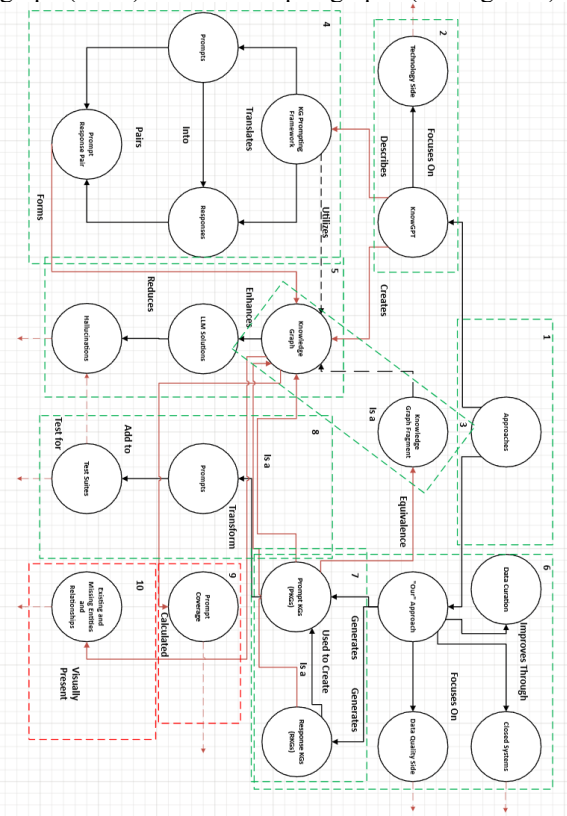

Figure 5. KG of KnowGPT Paragraphs.

Dashed edges represent the beginnings of knowledge graph completion, where existing relationships are examined for potential extensions. For instance, the graph identifies two approaches, KnowGPT and our own, and evaluates whether their relationships imply similar patterns across entities. Definitions, such as the connection between knowledge graph fragments and full knowledge graphs, are also incorporated. Importantly, because the RKG in this context supports prompt generation rather than system response, graph completion serves to expand test case variety rather than enrich outputs.

#	Prompt
1	How do the two knowledge graph approaches described in this paper differ in their goals and methods?
2	What area, technology or data, does the KnowGPT approach focus on?
3	What is the relationship between knowledge graph fragments and knowledge graphs?
4	Describe the KnowGPT approach.
5	What are the benefits of KnowGPT?
6	Describe how our approach improves data quality.
7	Describe the relationship between prompt knowledge graphs and response knowledge graphs.
8	Describe the evolution of prompts to test suites.
9	What is prompt coverage?
10	How do we discover existing and missing entities and relationships?

Table 1. Prompts based on Figure 5.

Within figure 5, the red boxes denote areas where prompts are likely to induce hallucinations or non-answers, while green boxes highlight regions expected to yield accurate responses. Using Step 7, these zones can be linked to predicted hallucination patterns. Step 8 then translates the graph nodes and edges into prompt knowledge graphs, which in turn generate concrete test prompts (see Table 1).

System Prompt: "You are an accurate paper reviewer for an upcoming AI conference."
Temperature = 0

#	Format (Bloom)	Expected Outcome (Section 10)	KG Entities (partial list)	Test Result
1	Analytical	Success	Approach	Pass
2	Fact Recall	Success	KnowGPT	Pass
3	Conceptual Understanding	Success	KG Fragments	Pass
4	Descriptive	Success	KnowGPT	Pass
5	Evaluation	OG	KnowGPT	Fail
6	Descriptive	Success	Our Approach	Pass
7	Conceptual Understanding	Success	PKG, RKG	Pass
8	Procedural	Success	Prompts	Pass
9	Conceptual Understanding	RF	Prompt Coverage	Fail
10	Procedural	RF	Entities	Fail

Table 2. Prompt information based on Table 1.

The prompt formats in Table 2 are based on Bloom's Taxonomy [Anderson].

Prompt Coverage = # of tested relationships (black)/ total # of relationships (red + black in Figure 5 including those from entities with no outgoing relationships) = 20/35 = 57%.

13. Conclusion

This paper introduced a framework that applies knowledge graphs to systematically detect

hallucinations in closed RAG-based and CustomGPT systems. Central to the approach is the concept of prompt coverage, an analog to software test coverage, which measures how thoroughly prompts exercise the system's knowledge and reasoning space. By aligning knowledge graph gaps with hallucination risks, we provide a new perspective on structured testing. Missing entities and relationships are not just data quality issues but predictive markers for where hallucinations are most likely to occur. The framework builds on existing knowledge to create ontologies and test cases, offering a principled way to probe systems that operate within finite, curated data boundaries.

Our method leverages LLMs' ability to generate knowledge graphs from documents and to perform partial knowledge graph completion checks, such as detecting missing relationships or nodes without outgoing edges. However, human-in-the-loop evaluation remains necessary to identify missing entities and to assess initial system responses. This hybrid approach enables scalable testing while retaining expert oversight in critical areas where automation alone is insufficient. Importantly, the framework has been applied to thousands of pages of documents used to build a CustomGPT ("AI Research") on several of the popular GPT platforms, demonstrating its feasibility at scale. The framework was able to correctly identify over two hundred prompts that generated hallucinations or "I don't know" answers (some technologies are better at handling the absence of information).

Solving the bounded problem of prompt coverage in closed systems with full access to curated data is the first step toward solving the broader and more difficult challenge of testing LLMs in general [Hobbhahn]. Open and hybrid systems introduce additional complexities, dynamic corpora, incomplete ontologies, and opaque system prompts that demand even more robust testing strategies. Further study is needed to automate greater portions of the knowledge-graph-based workflow, but the approach outlined here could extend existing automated evaluation frameworks, such as Evals, with structured, explainable methods for detecting hallucinations. In this way, the work presented offers both immediate practical value and a foundation for advancing the science of LLM testing.

14. Acknowledgements

The author would like to thank Jonathan Hart, John Peebles, Glen Johnson, Erik Peterson, Howard Gibbons, Sheri Fentress, Will Money, Zina Miller, and the HICSS reviewers for their feedback on this paper.

15. References

Anderson, L.W., & Krathwohl, D.R. (2001). A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives.

Armour, F. & Miller, G. (2001). Advanced Use Case Modeling, Addison Wesley.

Bouchard, L & Peters, L. (2024). Building LLMs for production: enhancing LLM abilities and reliability with prompting, fine-tuning, and RAG. Towards AI, Inc.

Chang, K., et al. (2021) "Data Curation Challenges for Artificial Intelligence." Auto-Segmentation for Radiation Oncology. CRC Press, 201-216.

Chen, Z., Wang, Y., Zhao, B., Cheng, J., Zhao, X. & Duan, Z. (2020). "Knowledge Graph Completion: A Review," in IEEE Access, vol. 8, pp. 192435-192456.

Devlin, Jacob, et al. (2019) "Bert: Pre-training of deep bidirectional transformers for language understanding." Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1. 2019,

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2025) "Ragas: Automated Evaluation of Retrieval Augmented Generation", https://arxiv.org/abs/2309.15217

Hobbhahn, M. "The Evals Gap", https://www.apolloresearch.ai/blog/evalsgap

Jiménez, L. (2025) "A Sociotechnical Approach for Knowledge Management (KM)." arXiv preprint, https://arxiv.org/abs/2502.06899.

Kejriwal, M., Knoblock, C. & Szekely, P. (2021). Knowledge graphs: Fundamentals, techniques, and applications, MIT Press.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). Survey of Hallucination in Natural Language Generation. ACM Computing Surveys (CSUR).

Sansford, H., Richardson, N., Maretic, H., & Saada, J. (2024) "GraphEval: A Knowledge-Graph Based LLM Hallucination Evaluation Framework". arXiv preprint arXiv:2407.10793, 2024

Shvaiko, P. & Euzenat, J. (2013). Ontology matching: state of the art and future challenges. IEEE Transactions on Knowledge and Data Engineering, 25 (1), pp.158-176.

Wang, Quan, Bin Wang, and Li Guo. (2015) "Knowledge Base Completion Using Embeddings and Rules." IJCAI.

Zhang, Q., Dong, J., Chen, H. Zha, D. Yu, Z. & Huang, X. (2024). KnowGPT: Knowledge Graph based PrompTing for Large Language Models. 38th Conference on Neural Information Processing Systems (NeurIPS 2024).

Zhao, W., Liu, J., Lin, Y., Zhou, Q., & Liu, Z. (2023). KGAug: Knowledge Graph Augmented Data for Factual Consistency in Natural Language Generation. ACL 2023.